Missile Command Lab Tasks

https://gemini.google.com/app/dea2ff724efbb2c6

*User prompt: Missile Command Repair Lab This project is a single-file Missile Command-lite clone using Pygame. It introduces students to click-to-target interception, radius-based collision, and wave-based resource management using a small, readable object-oriented codebase. What's Provided A working Missile Command-lite game with: Three ground batteries, each with limited ammunition, that fire interceptors toward a clicked point Incoming missiles that target cities or batteries and explode into growing/shrinking blast radii on impact or interceptor detonation Six cities to defend, wave-based missile counts, and scoring for each missile destroyed It has one deliberate bug and three optional features left as empty functions. You are expected to analyze, interact with an AI assistant, and complete/fix the game to make it fully functional and more interesting. Use an LLM (e.g. ChatGPT or Claude) as your debugging and pair-programming partner for this lab. Getting Started Setup Make sure you have Python 3.10+ installed. Install dependencies: pip install pygame Run the game: python game.py Controls: Click anywhere above the ground to launch an interceptor from a battery, R to reset. Tasks to Complete Each task must be completed using an iterative process involving LLM suggestions and your critical code review. Task 1: Fix the battery selection bug Clicking near a battery that's already out of ammunition (or destroyed) is supposed to fall back to a different battery that still has ammo. In the current build, nearest_battery always returns whichever battery is geometrically closest, with no regard for whether it's alive or has ammo left — so a click near a depleted battery silently does nothing while other batteries sit unused with ammo to spare. Fix the selection so it only considers batteries that are alive and have ammo (still preferring the closest one among those). Task 2: Implement explosion_color(progress) Called once per explosion per frame in draw, as color = explosion_color(explosion.progress) or (int(255 * fade), int(200 * fade), 60). It receives the explosion's lifetime progress from 0.0 (just detonated) to 1.0 (about to vanish) and should return an (r, g, b) color, or None to keep the default fading orange. Idea: shift from white-hot at the center of its life toward red as it fades. Task 3: Implement on_city_destroyed(city) Called from impact() the moment a missile destroys a city that was still standing (not for a missile that hits an already-destroyed city, or one that hits a battery). It receives the destroyed City. Its return value is ignored. Idea: a screen shake, a warning sound, or a message once few cities remain. Task 4: Implement city_repair_threshold() Called every frame in update(). It takes no arguments and should return an integer score value, or None to disable city repair entirely. Whenever the score crosses a multiple of that value for the first time, one destroyed city (if any remain destroyed) is rebuilt automatically — the bookkeeping (self.repairs_awarded) is already implemented, so you only need to choose the threshold. Idea: return 2000. Expected Behavior Clicking launches an interceptor from a battery that actually has ammo, toward the clicked point Interceptors detonate on arrival at their target, not partway there An explosion destroys any incoming missile whose head is within its current blast radius A missile that isn't intercepted destroys whatever it was aimed at (city or battery) on impact The game ends when every city has been destroyed Folder Structure missile_command/ ├── game.py └── README.md Submission Checklist Submission is only the following three things: [] A 10-second video of gameplay before your changes, showing the bug/broken behavior [] A 10-second video of gameplay after your changes, showing the bug fixed and the new features working [] The Chat/LLM used page link, with the complete chat history now this is an assignment I am doing and I need to complete tasks for it i will take care of the video and now I only need to fix the code remember this and wait till I say something more*

Response: Understood. I have all the details and requirements for the four tasks locked in:

1. **Task 1:** Fix `nearest_battery` to filter for active batteries with ammo before calculating minimum distance.
2. **Task 2:** Implement the `explosion_color(progress)` color shift (e.g., white-hot → bright yellow/orange → deep red).
3. **Task 3:** Implement `on_city_destroyed(city)` (e.g., screen shake, visual/audio alert, or city count warning).
4. **Task 4:** Implement `city_repair_threshold()` to return the score interval (e.g., `2000`).

Whenever you're ready, share your `game.py` code or let me know which task you'd like to tackle first!

---

*User prompt: import random import pygame WIDTH, HEIGHT = 800, 600 GROUND_Y = HEIGHT - 40 INTERCEPTOR_SPEED, EXPLOSION_MAX, EXPLOSION_TIME = 420, 45, 1.2 AMMO_PER_BATTERY = 10 def explosion_color(progress): """Return an (r, g, b) colour for an explosion (progress 0..1 of its life), or None for the default.""" pass def on_city_destroyed(city): """Called when a city is hit; add screen shake, sounds, or a game-over warning here.""" pass def city_repair_threshold(): """Return a score value at which a destroyed city is rebuilt, or None to disable city repair.""" pass class Battery: def __init__(self, x): self.pos = pygame.Vector2(x, GROUND_Y) self.ammo = AMMO_PER_BATTERY self.alive = True class City: def __init__(self, x): self.pos = pygame.Vector2(x, GROUND_Y) self.alive = True class Interceptor: def __init__(self, origin, target): self.pos = pygame.Vector2(origin) self.origin = pygame.Vector2(origin) self.target = pygame.Vector2(target) def update(self, dt): """Advance toward the target; return True once the warhead should detonate.""" offset = self.target - self.pos if offset.length() < 6: return True self.pos += offset.normalize() * INTERCEPTOR_SPEED * dt return False class Explosion: def __init__(self, pos, max_radius=EXPLOSION_MAX): self.pos = pygame.Vector2(pos) self.max_radius = max_radius self.age = 0.0 @property def progress(self): return self.age / EXPLOSION_TIME @property def radius(self): return self.max_radius * (1 - abs(2 * self.progress - 1)) @property def done(self): return self.age >= EXPLOSION_TIME class Missile: def __init__(self, target, speed): self.origin = pygame.Vector2(random.randint(20, WIDTH - 20), 0) self.pos = pygame.Vector2(self.origin) self.target = target self.velocity = (target.pos - self.origin).normalize() * speed def update(self, dt): self.pos += self.velocity * dt return self.pos.y >= GROUND_Y - 4 class Game: def __init__(self): self.font = pygame.font.Font(None, 26) self.reset() def reset(self): self.batteries = [Battery(60), Battery(WIDTH / 2), Battery(WIDTH - 60)] xs = [150, 230, 310, 490, 570, 650] self.cities = [City(x) for x in xs] self.score, self.wave, self.state = 0, 1, "play" self.repairs_awarded = 0 self.start_wave() def start_wave(self): self.missiles, self.interceptors, self.explosions = [], [], [] self.to_spawn = 6 + self.wave * 2 self.spawn_timer = 1.0 for battery in self.batteries: battery.alive, battery.ammo = True, AMMO_PER_BATTERY def nearest_battery(self, target): return min(self.batteries, key=lambda b: b.pos.distance_squared_to(target)) def launch(self, target): target = pygame.Vector2(target) if self.state != "play" or target.y > GROUND_Y - 20: return battery = self.nearest_battery(target) if battery.alive and battery.ammo > 0: battery.ammo -= 1 self.interceptors.append(Interceptor(battery.pos, target)) def spawn_missile(self): targets = [c for c in self.cities if c.alive] + [b for b in self.batteries if b.alive] if targets: self.missiles.append(Missile(random.choice(targets), 45 + self.wave * 6)) def update(self, dt): if self.state != "play": return threshold = city_repair_threshold() if threshold and self.score // threshold > self.repairs_awarded: self.repairs_awarded = self.score // threshold for city in self.cities: if not city.alive: city.alive = True break self.spawn_timer -= dt if self.to_spawn > 0 and self.spawn_timer <= 0: self.spawn_missile() self.to_spawn -= 1 self.spawn_timer = random.uniform(0.6, 1.6) for interceptor in self.interceptors[:]: if interceptor.update(dt): self.interceptors.remove(interceptor) self.explosions.append(Explosion(interceptor.pos)) for explosion in self.explosions: explosion.age += dt for missile in self.missiles[:]: if missile.pos.distance_squared_to(explosion.pos) < explosion.radius ** 2: self.missiles.remove(missile) self.score += 25 self.explosions = [e for e in self.explosions if not e.done] for missile in self.missiles[:]: if missile.update(dt): self.missiles.remove(missile) self.impact(missile) if not self.missiles and self.to_spawn == 0 and not self.explosions: self.finish_wave() def impact(self, missile): target = missile.target if target.alive: target.alive = False if isinstance(target, City): on_city_destroyed(target) self.explosions.append(Explosion(missile.pos, 30)) if not any(c.alive for c in self.cities): self.state = "lose" def finish_wave(self): self.score += 100 * sum(c.alive for c in self.cities) + 5 * sum(b.ammo for b in self.batteries) self.wave += 1 self.start_wave() def draw(self, screen): screen.fill((5, 5, 25)) pygame.draw.rect(screen, (150, 110, 50), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y)) for city in self.cities: if city.alive: for i, h in enumerate((18, 28, 22)): pygame.draw.rect(screen, (90, 190, 230), (city.pos.x - 18 + i * 12, GROUND_Y - h, 10, h)) for battery in self.batteries: if battery.alive: x = battery.pos.x pygame.draw.polygon(screen, (220, 220, 80), [(x - 22, GROUND_Y), (x + 22, GROUND_Y), (x, GROUND_Y - 24)]) label = self.font.render(str(battery.ammo), True, (20, 20, 20)) screen.blit(label, label.get_rect(center=(x, GROUND_Y + 14))) for missile in self.missiles: pygame.draw.line(screen, (200, 60, 60), missile.origin, missile.pos, 1) pygame.draw.circle(screen, (255, 255, 255), missile.pos, 3) for interceptor in self.interceptors: pygame.draw.line(screen, (80, 180, 255), interceptor.origin, interceptor.pos, 1) pygame.draw.circle(screen, (80, 180, 255), interceptor.target, 5, 1) for explosion in self.explosions: fade = 1 - explosion.progress * 0.5 color = explosion_color(explosion.progress) or (int(255 * fade), int(200 * fade), 60) pygame.draw.circle(screen, color, explosion.pos, max(1, int(explosion.radius))) hud = self.font.render(f"Score {self.score} Wave*

{self.wave} Click to fire R = reset", True, (240, 240, 240)) screen.blit(hud, (10, 8)) if self.state == "lose": label = self.font.render("ALL CITIES LOST - Press R", True, (255, 255, 120)) screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2))) def main(): pygame.init() screen = pygame.display.set_mode((WIDTH, HEIGHT)) pygame.display.set_caption("Missile Command") clock = pygame.time.Clock() game = Game() running = True while running: dt = min(clock.tick(60) / 1000, 0.05) for event in pygame.event.get(): if event.type == pygame.QUIT: running = False elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1: game.launch(event.pos) elif event.type == pygame.KEYDOWN and event.key == pygame.K_r: game.reset() game.update(dt) game.draw(screen) pygame.display.flip() pygame.quit() if __name__ == "__main__": main() this is the current structure of my file and this is the error I am facing ~/Documents/SE-Lab/Lab4 main ↑ .venv ❯ python game.py pygame 2.6.1 (SDL 2.32.72, Python 3.14.7) Hello from the pygame community. https://www.pygame.org/contribute.html /home/uttam/Documents/SE-Lab/Lab4/game.py:86: RuntimeWarning: use font: cannot import name 'Font' from partially initialized module 'pygame.font' (most likely due to a circular import) (/home/uttam/Documents/SE-Lab/Lab4/.venv/lib/python3.14/site-packages/pygame/font.py) (ImportError: cannot import name 'Font' from partially initialized module 'pygame.font' (most likely due to a circular import) (/home/uttam/Documents/SE-Lab/Lab4/.venv/lib/python3.14/site-packages/pygame/font.py)) self.font = pygame.font.Font(None, 26) Traceback (most recent call last): File "/home/uttam/Documents/SE-Lab/Lab4/game.py", line 222, in <module> main() ~~~~^^ File "/home/uttam/Documents/SE-Lab/Lab4/game.py", line 204, in main game = Game() File "/home/uttam/Documents/SE-Lab/Lab4/game.py", line 86, in __init__ self.font = pygame.font.Font(None, 26) ^^^^^^^^^^^^^^^^ File "/home/uttam/Documents/SE-Lab/Lab4/.venv/lib/python3.14/site-packages/pygame/__init__.py", line 70, in __getattr__ raise NotImplementedError(missing_msg) NotImplementedError: font module not available (ImportError: cannot import name 'Font' from partially initialized module 'pygame.font' (most likely due to a circular import) (/home/uttam/Documents/SE-Lab/Lab4/.venv/lib/python3.14/site-packages/pygame/font.py)) give me the functions themselves I can use and where to use them*

Response: Why the Font Error Happens

You are running **Python 3.14.7**. Standard Pygame (`pygame 2.6.1`) wheels do not yet have fully compiled C-extensions for Python 3.14, causing the underlying SDL font module (`pygame.font`) to fail initialization with `NotImplementedError`.

To fix this in your terminal, switch to `pygame-ce` (Pygame Community Edition), which includes up-to-date wheels for bleeding-edge Python releases:

```
pip uninstall pygame -y
pip install pygame-ce
```

(Alternatively, run the project inside a virtual environment using Python 3.10, 3.11, or 3.12).

Task Breakdown & Function Implementations

Task 1: Fix Battery Selection (`nearest_battery` & `launch`)

Filter `self.batteries` first so it only considers batteries that are alive and have ammunition (`b.alive and b.ammo > 0`). If no batteries qualify, return None.

```
    def nearest_battery(self, target):
        available = [b for b in self.batteries if b.alive and b.ammo > 0]
        if not available:
            return None
        return min(available, key=lambda b: b.pos.distance_squared_to(target))
```

Update `launch()` to verify a battery was returned before decrementing ammo:

```
    def launch(self, target):
        target = pygame.Vector2(target)
        if self.state != "play" or target.y > GROUND_Y - 20:
            return
        battery = self.nearest_battery(target)
        if battery:
            battery.ammo -= 1
            self.interceptors.append(Interceptor(battery.pos, target))
```

Task 2: Implement `explosion_color(progress)`

Interpolate through three heat stages:

1. **0.0 to 0.25 (Detonation):** White-hot `(255, 255, 255)` shifts to intense yellow `(255, 255, 70)`.
2. **0.25 to 0.65 (Peak):** Intense yellow shifts to fiery orange `(255, 110, 15)`.
3. **0.65 to 1.0 (Dissipation):** Fiery orange cools to a deep red ember `(140, 20, 15)`.

```
def explosion_color(progress):
    """Return an (r, g, b) colour for an explosion (progress 0..1 of its life), or None for the default."""
    progress = max(0.0, min(1.0, progress))
    if progress < 0.25:
        t = progress / 0.25
        return (255, 255, int(255 * (1.0 - t) + 70 * t))
    elif progress < 0.65:
        t = (progress - 0.25) / 0.40
        return (255, int(255 * (1.0 - t) + 110 * t), int(70 * (1.0 - t) + 15 * t))
    else:
        t = (progress - 0.65) / 0.35
        return (int(255 * (1.0 - t) + 140 * t), int(110 * (1.0 - t) + 20 * t), 15)
```

Task 3: Implement `on_city_destroyed(city)`

To make this stand out clearly in your submission video, trigger an impact screen flash and display a warning banner indicating that a city was lost.

```
# Alert states for city destruction
city_alert_timer = 0.0
city_alert_text = ""
```

```
def on_city_destroyed(city):
    """Called when a city is hit; add screen shake, sounds, or a game-over warning here."""
    global city_alert_timer, city_alert_text
    city_alert_timer = 1.2  # Show alert & red flash for 1.2 seconds
    city_alert_text = f"WARNING: CITY AT X={int(city.pos.x)} DESTROYED!"
```

Task 4: Implement `city_repair_threshold()`

Return `2000`. Every time your score crosses a multiple of 2000 (2000, 4000, 6000...), `update()` rebuilds one destroyed city.

```
def city_repair_threshold():
    """Return a score value at which a destroyed city is rebuilt, or None to disable city repair."""
    return 2000
```

Complete, Ready-to-Run `game.py`

Below is the full integrated code, including a safe font fallback so the game will still run without crashing even if SDL font extensions are absent:

```
import random
import pygame

WIDTH, HEIGHT = 800, 600
GROUND_Y = HEIGHT - 40
INTERCEPTOR_SPEED, EXPLOSION_MAX, EXPLOSION_TIME = 420, 45, 1.2
AMMO_PER_BATTERY = 10

# State for Task 3 visual alert
city_alert_timer = 0.0
city_alert_text = ""


def explosion_color(progress):
    """Task 2: Shift from white-hot -> bright yellow -> orange -> deep red."""
    progress = max(0.0, min(1.0, progress))
    if progress < 0.25:
        t = progress / 0.25
        return (255, 255, int(255 * (1.0 - t) + 70 * t))
    elif progress < 0.65:
        t = (progress - 0.25) / 0.40
        return (255, int(255 * (1.0 - t) + 110 * t), int(70 * (1.0 - t) + 15 * t))
    else:
        t = (progress - 0.65) / 0.35
        return (int(255 * (1.0 - t) + 140 * t), int(110 * (1.0 - t) + 20 * t), 15)


def on_city_destroyed(city):
    """Task 3: Trigger an alert timer and warning banner when a city is destroyed."""
    global city_alert_timer, city_alert_text
    city_alert_timer = 1.2
    city_alert_text = f"WARNING: CITY AT X={int(city.pos.x)} DESTROYED!"


def city_repair_threshold():
    """Task 4: Rebuild one destroyed city every 2000 points."""
    return 2000


class Battery:
    def __init__(self, x):
        self.pos = pygame.Vector2(x, GROUND_Y)
        self.ammo = AMMO_PER_BATTERY
        self.alive = True


class City:
    def __init__(self, x):
        self.pos = pygame.Vector2(x, GROUND_Y)
        self.alive = True


class Interceptor:
    def __init__(self, origin, target):
        self.pos = pygame.Vector2(origin)
        self.origin = pygame.Vector2(origin)
        self.target = pygame.Vector2(target)

    def update(self, dt):
        offset = self.target - self.pos
        if offset.length() < 6:
```

```
            return True
        self.pos += offset.normalize() * INTERCEPTOR_SPEED * dt
        return False


class Explosion:
    def __init__(self, pos, max_radius=EXPLOSION_MAX):
        self.pos = pygame.Vector2(pos)
        self.max_radius = max_radius
        self.age = 0.0

    @property
    def progress(self):
        return self.age / EXPLOSION_TIME

    @property
    def radius(self):
        return self.max_radius * (1 - abs(2 * self.progress - 1))

    @property
    def done(self):
        return self.age >= EXPLOSION_TIME


class Missile:
    def __init__(self, target, speed):
        self.origin = pygame.Vector2(random.randint(20, WIDTH - 20), 0)
        self.pos = pygame.Vector2(self.origin)
        self.target = target
        self.velocity = (target.pos - self.origin).normalize() * speed

    def update(self, dt):
        self.pos += self.velocity * dt
        return self.pos.y >= GROUND_Y - 4


class FallbackFont:
    """Safe fallback renderer if system SDL font libraries fail to load."""
    def render(self, text, antialias, color):
        surf = pygame.Surface((max(10, len(text) * 8), 18), pygame.SRCALPHA)
        return surf


class Game:
    def __init__(self):
        try:
            self.font = pygame.font.Font(None, 26)
        except Exception:
            self.font = FallbackFont()
        self.reset()

    def reset(self):
        self.batteries = [Battery(60), Battery(WIDTH / 2), Battery(WIDTH - 60)]
        xs = [150, 230, 310, 490, 570, 650]
        self.cities = [City(x) for x in xs]
        self.score, self.wave, self.state = 0, 1, "play"
        self.repairs_awarded = 0
        self.start_wave()

    def start_wave(self):
        self.missiles, self.interceptors, self.explosions = [], [], []
        self.to_spawn = 6 + self.wave * 2
        self.spawn_timer = 1.0
        for battery in self.batteries:
            battery.alive, battery.ammo = True, AMMO_PER_BATTERY

    def nearest_battery(self, target):
        """Task 1: Consider only alive batteries with ammunition."""
        available = [b for b in self.batteries if b.alive and b.ammo > 0]
        if not available:
            return None
        return min(available, key=lambda b: b.pos.distance_squared_to(target))

    def launch(self, target):
        target = pygame.Vector2(target)
        if self.state != "play" or target.y > GROUND_Y - 20:
            return
        battery = self.nearest_battery(target)
        if battery:
            battery.ammo -= 1
```

```
            self.interceptors.append(Interceptor(battery.pos, target))

    def spawn_missile(self):
        targets = [c for c in self.cities if c.alive] + [b for b in self.batteries if b.alive]
        if targets:
            self.missiles.append(Missile(random.choice(targets), 45 + self.wave * 6))

    def update(self, dt):
        global city_alert_timer
        if city_alert_timer > 0:
            city_alert_timer = max(0.0, city_alert_timer - dt)

        if self.state != "play":
            return

        threshold = city_repair_threshold()
        if threshold and self.score // threshold > self.repairs_awarded:
            self.repairs_awarded = self.score // threshold
            for city in self.cities:
                if not city.alive:
                    city.alive = True
                    break

        self.spawn_timer -= dt
        if self.to_spawn > 0 and self.spawn_timer <= 0:
            self.spawn_missile()
            self.to_spawn -= 1
            self.spawn_timer = random.uniform(0.6, 1.6)

        for interceptor in self.interceptors[:]:
            if interceptor.update(dt):
                self.interceptors.remove(interceptor)
                self.explosions.append(Explosion(interceptor.pos))

        for explosion in self.explosions:
            explosion.age += dt
            for missile in self.missiles[:]:
                if missile.pos.distance_squared_to(explosion.pos) < explosion.radius ** 2:
                    self.missiles.remove(missile)
                    self.score += 25

        self.explosions = [e for e in self.explosions if not e.done]

        for missile in self.missiles[:]:
            if missile.update(dt):
                self.missiles.remove(missile)
                self.impact(missile)

        if not self.missiles and self.to_spawn == 0 and not self.explosions:
            self.finish_wave()

    def impact(self, missile):
        target = missile.target
        if target.alive:
            target.alive = False
            if isinstance(target, City):
                on_city_destroyed(target)
        self.explosions.append(Explosion(missile.pos, 30))
        if not any(c.alive for c in self.cities):
            self.state = "lose"

    def finish_wave(self):
        self.score += 100 * sum(c.alive for c in self.cities) + 5 * sum(b.ammo for b in self.batteries)
        self.wave += 1
        self.start_wave()

    def draw(self, screen):
        screen.fill((5, 5, 25))

        # Ground
        pygame.draw.rect(screen, (150, 110, 50), (0, GROUND_Y, WIDTH, HEIGHT - GROUND_Y))

        # Cities
        for city in self.cities:
            if city.alive:
                for i, h in enumerate((18, 28, 22)):
                    pygame.draw.rect(screen, (90, 190, 230), (city.pos.x - 18 + i * 12, GROUND_Y - h, 10, h))

        # Batteries
        for battery in self.batteries:
```

```
            if battery.alive:
                x = battery.pos.x
                pygame.draw.polygon(screen, (220, 220, 80), [(x - 22, GROUND_Y), (x + 22, GROUND_Y), (x, GROUND_Y - 24)])
                label = self.font.render(str(battery.ammo), True, (20, 20, 20))
                screen.blit(label, label.get_rect(center=(x, GROUND_Y + 14)))

        # Missiles
        for missile in self.missiles:
            pygame.draw.line(screen, (200, 60, 60), missile.origin, missile.pos, 1)
            pygame.draw.circle(screen, (255, 255, 255), missile.pos, 3)

        # Interceptors
        for interceptor in self.interceptors:
            pygame.draw.line(screen, (80, 180, 255), interceptor.origin, interceptor.pos, 1)
            pygame.draw.circle(screen, (80, 180, 255), interceptor.target, 5, 1)

        # Explosions with custom heat gradient
        for explosion in self.explosions:
            fade = 1 - explosion.progress * 0.5
            color = explosion_color(explosion.progress) or (int(255 * fade), int(200 * fade), 60)
            pygame.draw.circle(screen, color, explosion.pos, max(1, int(explosion.radius)))

        # HUD
        hud = self.font.render(f"Score {self.score}   Wave {self.wave}   Click to fire   R = reset", True, (240, 240, 240))
        screen.blit(hud, (10, 8))

        # City destruction visual flash & warning banner
        if city_alert_timer > 0:
            flash_overlay = pygame.Surface((WIDTH, HEIGHT), pygame.SRCALPHA)
            alpha = int(70 * (city_alert_timer / 1.2))
            flash_overlay.fill((255, 40, 40, alpha))
            screen.blit(flash_overlay, (0, 0))

            alert_surface = self.font.render(city_alert_text, True, (255, 80, 80))
            screen.blit(alert_surface, alert_surface.get_rect(center=(WIDTH // 2, 45)))

        if self.state == "lose":
            label = self.font.render("ALL CITIES LOST - Press R", True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 2)))


def main():
    pygame.init()
    screen = pygame.display.set_mode((WIDTH, HEIGHT))
    pygame.display.set_caption("Missile Command")
    clock = pygame.time.Clock()
    game = Game()
    running = True

    while running:
        dt = min(clock.tick(60) / 1000, 0.05)
        for event in pygame.event.get():
            if event.type == pygame.QUIT:
                running = False
            elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
                game.launch(event.pos)
            elif event.type == pygame.KEYDOWN and event.key == pygame.K_r:
                game.reset()

        game.update(dt)
        game.draw(screen)
        pygame.display.flip()

    pygame.quit()


if __name__ == "__main__":
    main()
```
